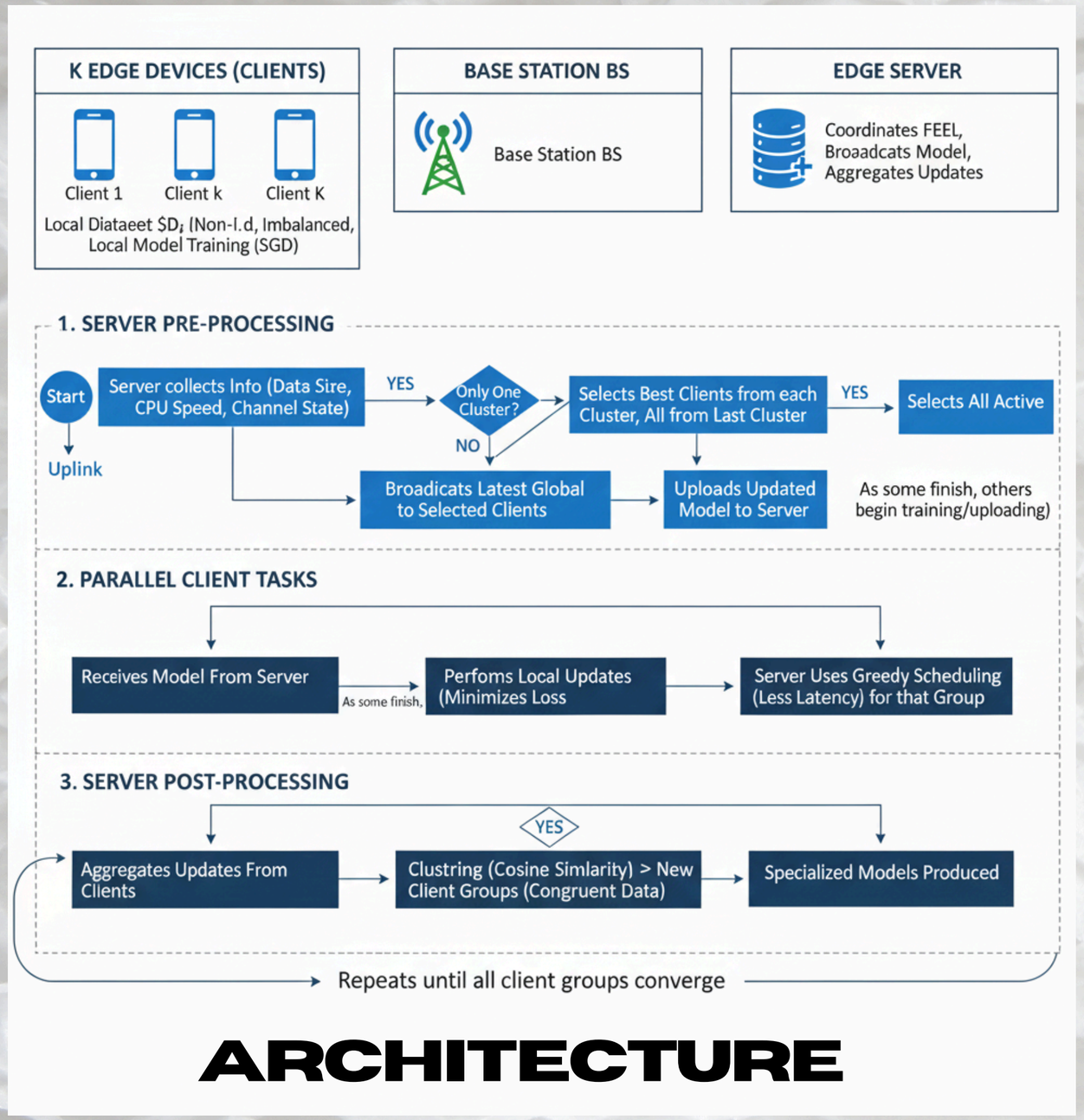
Architecture | Progress | Demonstration | Results | Configuration | Edge device | Edge Server | Algorithms | To be Done

Team - 23_XpertEdge

Client Selection Approach in Support of Clustered Federated Learning over Wireless Edge Networks

REVIEW - 2



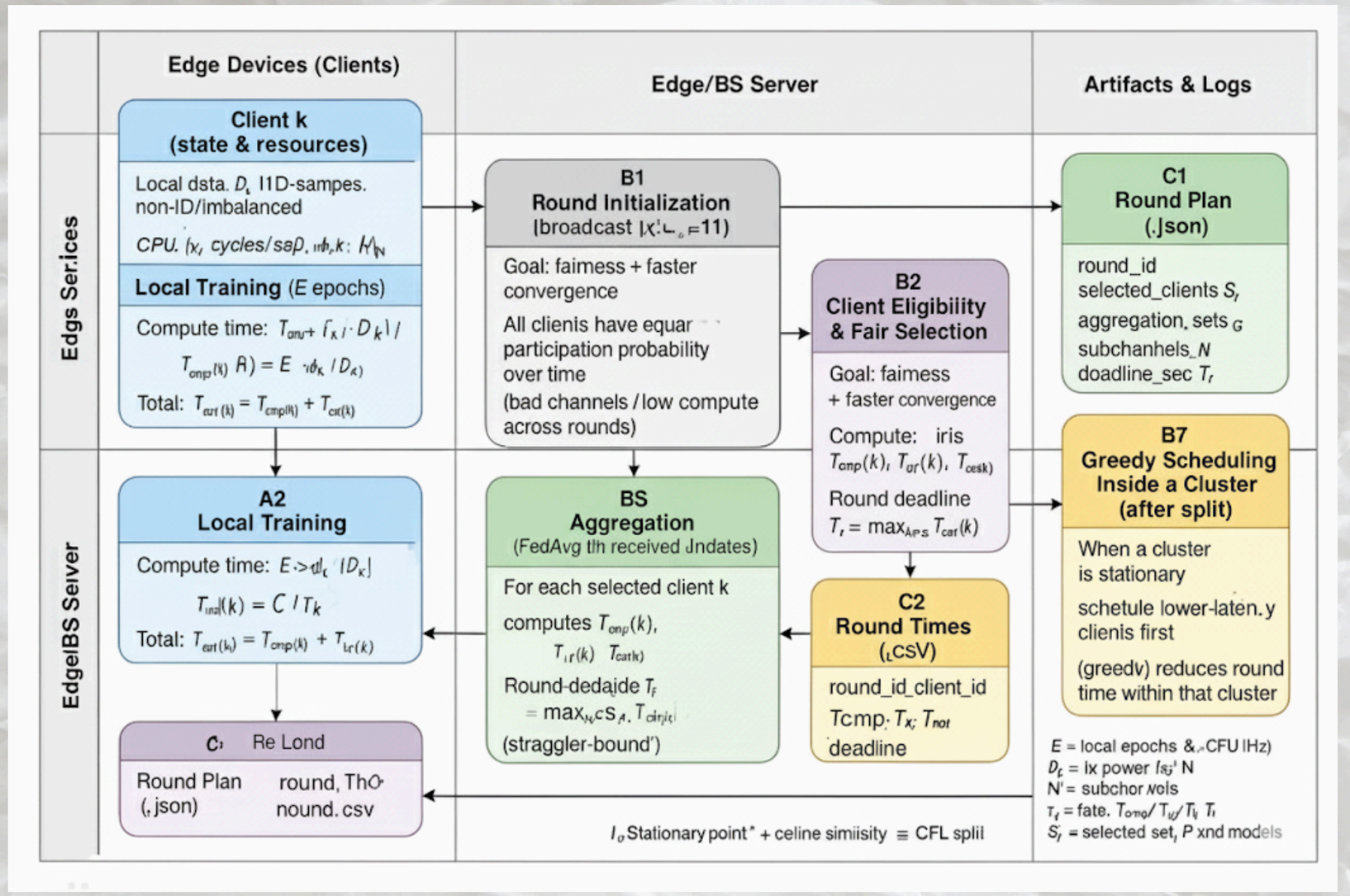
ARCHITECTURE

PROBLEM STATEMENRT

THE CORE PROBLEM IS THAT IMPLEMENTING CLUSTERED FEDERATED LEARNING (CFL) IN WIRELESS EDGE NETWORKS IS CHALLENGING DUE TO LIMITED BANDWIDTH AND LATENCY CONSTRAINTS, WHICH PREVENT THE SERVER FROM INVOLVING ALL CLIENTS IN EACH TRAINING ROUND.

PROGRESS FROM REVIEW 1 (75%)

- **Simulated device Configurations**
- **Created Client Architecture**
- **Created Server Architecture**
- **BaseLine Client Scheduling Algorithm**


Edge Devices (Clients)
Edge/BS Server
Artifacts & Logs
Edgs Ser.ices
EdgelBS Seiver
Client k
(state & resources)
Local dsta. D_k I1D-sampes.
non-ID/imbalanced
Local Training (E epochs)
Compute time:
Total:
B1
Round Initialization
Goal: faimess + faster convergence
All clienis have equar participation probability over time
(bad channels / low compute across rounds)
B2
Client Eligibiiity & Fair Selection
Goal: faimess + faster convergence
Compute: iris
Round deadline
C1
Round Plan
(.Json)
round_id
selected_clients S_t
aggregation. sets G
subchanhels_N
doadline_sec T_t
B7
Greedy Scheduling Inside a Cluster (after split)
When a cluster is stationary
schetule lower-laten.y clienis first
(greedv) reduces round time within that cluster
A2
Local Training
Compute time:
Total:
BS
Aggregation
(FedAvg tlh received Jndates)
For each selected client k
computes
Round-dedaide T_F
(straggler-bound')
C2
Round Times
(LCSV)
round_id_client_id
deadline
C1 Re Lond
Round Plan (, json)
round, ThO· nound. csv
Stationary point * + celine simiisity ≡ CFL spiil
E = local epochs &,·CFU IHz)
D_t = Ix power
N' = subchor nels
S_t = selected set, P xnd models

```
{
  "round_id": 1,
  "selected_clients": [
    {
      "id": "client_f0451_44",
      "cluster": "base"
    },
    {
      "id": "client_f0017_07",
      "cluster": "base"
    },
    {
      "id": "client_f0379_10",
      "cluster": "base"
    },
    {
      "id": "client_f0360_15",
      "cluster": "base"
    },
    {
      "id": "client_f0863_41",
      "cluster": "base"
    },
    {
      "id": "client_f0035_19",
      "cluster": "base"
    },
    {
      "id": "client_f1301_47",
      "cluster": "base"
    },
    {
      "id": "client_f0246_12",
      "cluster": "base"
    },
    {
      "id": "client_f1369_36",
      "cluster": "base"
    }
  ],
```


Round Information


```
"aggregation_sets": [
    [
      "client_f0451_44",
      "client_f0017_07",
      "client_f0379_10"
    ],
    [
      "client_f0360_15",
      "client_f0863_41",
      "client_f0035_19"
    ],
    [
      "client_f1301_47",
      "client_f0246_12",
      "client_f1369_36"
    ]
  ],
  "subchannels_N": 3,
  "deadline_sec": 26111588.632582925
```


Cluster Formation

```
[CLIENT client_f0070_14] Tcmp=0.000013 s, Ttx=896125.905853 s, Ttot=896125.905866 s

[FUNC achievable_rate_bps] cid=client_f0017_07, bw=3.33e+06 Hz, noise=1.00e-06 W -> rate=2.830849e+01 bps

[FUNC tx_time_sec] model_size=1200000 bytes, rate=2.830849e+01 bps -> time=42390.111524 s

[FUNC compute_time_sec] E=10, cycles/sample=20, data_size=424, cpu=2.97e+09 Hz -> time=0.000029 s
```

Device Information

```
Round 002 summary:
  Selected clients:   9
  Round deadline:     3476887.462 s
  Aggregation sets:   3

Round 002: ML updates skipped (dry run)

select_random -> ['client_f0054_14', 'client_f0255_48', 'client_f0099_18', 'client_f0155_13', 'client_f0261_06', 'client_f0115_34', 'client_f0023_08', 'client_f0256_08',
'client_f0613_28']
build_aggregation_sets -> [['client_f0054_14', 'client_f0255_48', 'client_f0099_18'], ['client_f0155_13', 'client_f0261_06', 'client_f0115_34'], ['client_f0023_08', 'clie
nt_f0256_08', 'client_f0613_28']]

[FUNC achievable_rate_bps] cid=client_f0054_14, bw=3.33e+06 Hz, noise=1.00e-06 W -> rate=5.170386e+00 bps

[FUNC tx_time_sec] model_size=1200000 bytes, rate=5.170386e+00 bps -> time=232090.971762 s

[FUNC compute_time_sec] E=10, cycles/sample=20, data_size=428, cpu=1.81e+09 Hz -> time=0.000047 s


[CLIENT client_f0054_14] Tcmp=0.000047 s, Ttx=232090.971762 s, Ttot=232090.971810 s


[FUNC achievable_rate_bps] cid=client_f0255_48, bw=3.33e+06 Hz, noise=1.00e-06 W -> rate=4.487891e-02 bps

[FUNC tx_time_sec] model_size=1200000 bytes, rate=4.487891e-02 bps -> time=26738616.483232 s

[FUNC compute_time_sec] E=10, cycles/sample=20, data_size=418, cpu=3.46e+09 Hz -> time=0.000024 s
```

Round Information

CONFIG

Python Code (SimDirs, SimConfig)

This code's job is to set up the simulation.

- **SimDirs** just keeps track of the output folder paths.
- **SimConfig** holds all the simulation settings like the number of rounds, bandwidth, and CPU speeds.
- **The load_sim_config()** function reads a YAML file, creates these objects, sets a random seed for reproducible results, and makes sure all the necessary folders exist before the simulation begins.

YAML Configuration File

This is a simple text file that contains all the simulation's settings in an easy-to-read format.

- **General: seed: 42** ensures the simulation runs the same way every time.
- **System:** Sets up the wireless environment, including **bandwidth_MHz** (total radio bandwidth) and noise_watt (background noise).
- **Path loss model:** Defines how the signal gets weaker with distance. It includes the **distance_m_range** for client placement and the **pathloss_exponent.**
- **Compute model:** Specifies the device capabilities, like **cpu_freq_GHz_range** (CPU speeds) and how many **local_epochs_E** each device trains for.
- **Transmission power:** Sets the **tx_power_dBm_range** for client devices.
- **Training round control:** Determines the number of **rounds_R** and the **max_clients_per_round.**
- **Orchestrator behavior:** Controls how the central server manages the training process, including waiting for all updates (**block_on_ml_updates)** and setting a **ml_updates_timeout_sec.**

Client

1 ClientProfile (Dataclass)

- Represents a client device with dataset size,distance,CPU frequency,channel gain, and transmission power.
- Stores device-specific properties needed for training and communication.

2 dbm_to_watt(dbm)

- Converts transmission power from dbm to watts
- Formula- $P(W) = 10^{\frac{(dBm-30)}{10}}$.

3 dbm_to_lin(db)

- Converts decimal values into linear scale for calculations
- Formula- $lin = 10^{\frac{dB}{10}}$

4 _pathloss_gain_linear(g0_dB, d0, d, alpha)

- Computes wireless channel gain using pathloss model.
- Formula - $g = g0 \cdot (d0/d)^{\alpha}$

5 build_clients_from_sizes(data_sizes, cfg)

- Builds a dictionary of ClientProfile objects for all clients.
- Randomly assigns each client's distance, CPU frequency, transmission power, and channel gain based on simulation config.

KEY FUNCTIONS FOR CLIENT TRAINING & COMMUNICATION

1. **achievable_rate_bps(c, bw, noise)**
   - Calculates achievable data rate (bps) using Shannon's formula.
   - Inputs: Power, channel gain, bandwidth, noise.
2. **tx_time_sec(model_bytes, rate_bps)**
   - Transmission time = Model Size / Data Rate.
   - Inputs: Model size & achievable rate → seconds.
3. **compute_time_sec(E, cycles_per_sample, data_size, cpu_freq)**
   - Local training time = (Epochs × Cycles/Sample × Data Size) / CPU Freq.
   - Inputs: Epochs, dataset size, CPU speed → seconds.
4. **estimate_times_for_selected(selected, model_bytes, cfg)**
   - Estimates per-client times: computation, transmission, total.
   - Ensures fair bandwidth.
   - Tracks slowest client → max round processing time.

Edge – server

- **_read_shared_inputs** – Reads the model size and dataset sizes of all clients, so the server knows how much data each client has before selection.
- **_append_round_log** – Records each client's computation time, transmission time, total time, and writes these into a csv file.
- **_wait_for_ml_updates** – Makes the server wait for clients' local model updates, keeps checking at intervals and raises timeout error if no updates happen in time.
- **run_edge** – The main function it selects eligible clients, groups them, estimates their deadlines, writes a round plan to json,and logs the result into a csv,printing a summary and wait for ml updates.

Baseline Scheduler Algorithm

- **What it does**: Picks the round's participants uniformly at random and splits them into OFDMA batches (aggregation sets) of size ≤ subchannels_N for parallel uploads.
- **Inputs:** eligible_ids, max_clients_per_round, subchannels_N (RNG seeded via config).
- **Flow**: select_random() → choose unique clients without replacement → build_aggregation_sets() → chunk into parallel groups → (then latency module computes Tcmp, Ttx, Ttot, and the round deadline).
- **Outputs**: Writes round_{r}_plan.json with selected_clients, aggregation_sets, subchannels_N, deadline_sec; logs per-client times to round_times.csv.
- **Limitations (baseline)**: No fairness or latency-aware ordering yet—upgrade later with fair scheduler and CFL integration for cluster-aware selection.

WORK TO BE DONE (25 %)

- INTEGRATING CLUSTERED FEDERATED LEARNING (CFL) WITH CI
- INTEGRATION OF INDIVIDUAL MODULES
- FAIRNESS AWARE SCHEDULING ALGORITHM
- FINAL TESTING AND DEMONSTRATION

CONVOLUTIONAL NEURAL NETWORK

Why CNN?

- **Ideal for the FEMNIST Dataset:** CNNs are designed to process the image-based data in the FEMNIST dataset, automatically learning the features of letters and digits to achieve high classification accuracy.

- **Perfect for Federated Learning:** Modern CNN architectures can be made extremely lightweight and computationally efficient, which is essential for training on the resource-constrained edge devices used in a federated learning network.

Hyperparameters Tuned:

- Model Width Multiplier
- Learning Rate
- Batch Size
- Dropout Rate
- Weight Decay
- Optimizer Momentum
- Convolutional Layer Channels

GENETIC ALGORITHM BASED HYPERPARAM OPTIMISATION

Why GA?

- CNN performance is highly sensitive to hyperparameters (learning rate, batch size, dropout, weight decay, momentum, etc.).
- Manual tuning or random search is slow and may miss good combinations.
- GA provides a guided, evolutionary search to efficiently explore and find the optimal hyperparams

How It Works?

1. Population Creation
2. Fitness Evaluation
3. Parent Selection
4. Crossover
5. Mutation
6. Repeat for multiple generations

So, did it work?

Base CNN: 80.2 % test accuracy
GA optimised CNN: **86.38% test accuracy.**

```
Final Test: 100%|██████████| 1273/1273 [00:23<00:00, 54.97it/s]

Results:
Final Test Accuracy: 0.8638
Model saved as 'ga_optimized_model.pth'
```

CNN | Genetic Algorithm | Team Split | Q & A

MOHANA KRISHNAN	**BASELINE SCHEDULER,TESTING,WORK FLOW**
AMAL GODWIN	**CLIENT ARCHITECTURE**
SAURAV GOPINATH	**SERVER ARCHITECTURE**
DEEPAN	**CLIENT – SERVER CONFIGURATIONS**
ADITHYA MENON R	**Genetic Algorithm based Hyperparam Optimisation**
NARAIN BK	**Convolutional Neural Network**

THANK YOU

Q & A